

Mini-projet 01 — Parser une liste de cibles

Module A · Notions : **split** · **strip** · **slicing** · **compréhensions (liste & dict)** · Durée estimée : 1 h 30 – 2 h · Parcours Python — Ednir

Pourquoi ce mini-projet ?

Dans ta future boîte à outils, avant de scanner ou d'auditer quoi que ce soit, il faut **lire un fichier de cibles** (une `ip:port` par ligne) et le transformer en données Python exploitables. Transformer du texte brut en structures propres, c'est **la compétence de base** de tout outil. On la travaille ici, isolée, pour qu'elle rentre bien.

1. Les chaînes et leurs méthodes

1.1 — `.split()` : découper une chaîne

`.split(sep)` découpe une chaîne à chaque occurrence de `sep` et renvoie une **liste**.

```
ligne = "192.168.1.10:22"
morceaux = ligne.split(":")
# morceaux vaut -> ['192.168.1.10', '22']
print(morceaux[0]) # 192.168.1.10 (l'IP)
print(morceaux[1]) # 22 (le port, MAIS c'est une str !)
```

Piège n°1 — type du port. Ce que `split` renvoie, ce sont **toujours des chaînes**. Donc `morceaux[1]` vaut `"22"` (texte), pas `22` (nombre). Et attention :

`"22" == 22` renvoie `False` — une str et un int ne sont jamais égaux.

Si tu veux comparer ou trier des ports, convertis : `int("22")` → `22`.

1.2 — `.strip()` : nettoyer les bords

Quand tu lis un fichier, chaque ligne traîne souvent un **retour à la ligne** invisible (`\n`) ou des espaces. `.strip()` enlève les espaces et sauts de ligne **au début et à la fin**.

```
ligne = " 192.168.1.10:22\n"
ligne.strip() # -> '192.168.1.10:22' (propre)
# variantes : .lstrip() (gauche seulement), .rstrip() (droite seulement)
```

Réflexe : presque toujours, on écrit `ligne.strip().split(":")` — on nettoie d'abord, on découpe ensuite.

2. Le slicing (les tranches)

Le slicing extrait une **portion** d'une liste (ou d'une chaîne) avec la syntaxe `[début:fin]`. La borne **fin est exclue**.

```
nums = [10, 20, 30, 40, 50]
nums[0] # 10 premier élément
nums[-1] # 50 dernier élément (indices négatifs = depuis la fin)
nums[1:3] # [20, 30] du 1 inclus au 3 exclu
nums[:2] # [10, 20] du début jusqu'au 2 exclu
nums[2:] # [30, 40, 50] du 2 jusqu'à la fin
```

3. Les compréhensions de liste notion clé

C'est **le** gros morceau, et ton principal trou. Une compréhension = écrire une boucle qui construit une liste, mais **en une seule ligne**.

3.1 — Le même code, deux écritures

La version « classique » que tu connais déjà :

```
# On veut les carrés de 0 à 4
carrés = []
for x in range(5):
    carrés.append(x * x)
```

```
# carres -> [0, 1, 4, 9, 16]
```

La même chose en compréhension :

```
carres = [x * x for x in range(5)]  
# carres -> [0, 1, 4, 9, 16] exactement pareil, en 1 ligne
```

3.2 — L'anatomie (à mémoriser)

```
[ expression for variable in itérable ]
```

- **expression** : ce que tu mets dans la liste à chaque tour (`x * x`)
- **variable** : le nom qui prend chaque valeur (`x`)
- **itérable** : ce sur quoi on boucle (`range(5)` , une liste, un fichier...)

Lis-la à voix haute : « *x fois x, pour chaque x dans range(5)* ». L'ordre est inversé par rapport à la boucle : d'abord le résultat, ensuite le parcours.

3.3 — Avec un filtre (le `if`)

On peut ne garder que certains éléments en ajoutant un `if` à la fin :

```
# Seulement les nombres pairs de 0 à 9  
pairs = [x for x in range(10) if x % 2 == 0]  
# pairs -> [0, 2, 4, 6, 8]  
  
# Ignorer les lignes vides d'une liste de lignes  
lignes = ["a", "", "b", " "]  
propres = [l for l in lignes if l.strip()]  
# propres -> ['a', 'b'] (une chaîne vide est "fausse" pour if)
```

4. Les compréhensions de dictionnaire

Même idée, mais on construit un **dictionnaire** : on écrit `clé: valeur` à la place de l'expression, avec des accolades `{ }`.

```
mots = ["scan", "port", "ip"]  
longueurs = {mot: len(mot) for mot in mots}  
# longueurs -> {'scan': 4, 'port': 4, 'ip': 2}
```

5. Rappel : lire un fichier

Piège n°2 — l'objet fichier n'est pas le texte. `open("cibles.txt")` ne te donne **pas** le contenu : il te donne un **objet fichier** (une sorte de « robinet »). Pour obtenir les lignes, il faut le parcourir ou le lire.

```
with open("cibles.txt", encoding="utf-8") as f:  
    for ligne in f: # on boucle sur l'objet fichier -> ligne par ligne  
        print(ligne.strip())  
# le "with" ferme le fichier tout seul à la fin. On le verra en détail au mini-projet 13.
```

Ta mission

Objectif : écrire un script `parse_cibles.py` qui lit un fichier de cibles et en fait une liste de dictionnaires propres.

1. Crée à la main un fichier `cibles.txt` dans le même dossier, avec quelques lignes, par exemple :

```
192.168.1.10:22  
192.168.1.10:80  
10.0.0.5:443
```

2. Dans `parse_cibles.py` :

- ouvre et lis le fichier ligne par ligne ;
- pour chaque ligne, sépare l'**IP** et le **port** ;
- construis une liste de dictionnaires de la forme

`[{"ip": "192.168.1.10", "port": 22}, ...]` — avec le **port en `int`** ;

- affiche chaque cible proprement, par ex. `192.168.1.10 -> port 22` .

Contraintes (le cœur de l'exercice) :

- utilise **au moins une compréhension de liste** pour construire ta liste de cibles ;
- le port doit être un **entier**, pas une chaîne ;

Bonus (si tu es à l'aise) :

- ignore les **lignes vides** ;
- ignore les lignes de **commentaire** qui commencent par `#` ;
- ajoute une compréhension de **dictionnaire** qui compte, pour chaque IP, son nombre de ports.

✓ Critères de réussite

- Le script tourne sans erreur sur ton `cibles.txt` .
- `type(cible["port"])` renvoie `<class 'int'>` .
- Il y a une compréhension de liste dans ton code.
- (Bonus) une ligne vide ou commentée ne casse pas le script.

Si tu bloques — indices progressifs

Ne lis l'indice suivant que si le précédent ne t'a pas débloqué. L'idée, c'est que tu cherches d'abord.

Étape	Indice
Lire le fichier	Boucle <code>for ligne in f:</code> après un <code>with open(...) as f:</code> .
Séparer ip / port	<code>ip, port = ligne.strip().split(":")</code> — le « dépaquetage » à gauche marche parce que <code>split</code> renvoie exactement 2 morceaux.
Port en int	Là où tu construis le dict : <code>"port": int(port)</code> .
Compréhension	Récupère d'abord toutes les lignes, puis : <code>cibles = [ligne_vers_cible(l) for l in lignes]</code> (tu peux faire une petite fonction qui transforme UNE ligne en dict).
Ignorer vides / #	Ajoute un filtre : <code>... for l in lignes if l.strip() and not l.strip().startswith("#")</code> .

Quand tu as fini : commite ton travail dans le repo, puis envoie-moi juste le lien GitHub du dossier (ou un « c'est poussé »). Je lis ton code directement, je te dis ce qui est bien et ce qu'on affine — et on passe au mini-projet 02.

Rappel du cycle Git (dans le dossier du repo) :

`git add .` → `git commit -m "Mini-projet 01 : parsing de cibles"` → `git push`